计算机学院 并行程序设计实验报告

Lab2 SIMD 编程：ANN 近似最近邻搜索

AVX2 / NEON 向量化、量化检索与 FastScan 扩展

柯云超 2413575 计算机科学与技术 2026 年 4 月 19 日

摘要

本文围绕 ANN（Approximate Nearest Neighbor）选题完成 SIMD 编程实验，在 ARM NEON 与 x86 AVX2 平台上实现并比较了四类检索方法：串行 Flat、Flat-SIMD、SQ-SIMD、PQ-SIMD，并在 Windows i9-13900H 与鲲鹏 920 上进一步实现了基于 4-bit PQ 的 FastScan 扩展。针对 SIMD 编程实验的基础要求，本文不仅给出向量化算法设计与实现，还结合已有实验数据讨论了内存对齐、软件预取、问题规模变化、自动向量化与手写 SIMD、以及 VTune / 汇编层面的性能原因。实验使用 DEEP100K 数据集 [2]（$N = 100000$，$d = 96$，IP 距离），统一以前 2000 个查询、$k = 10$ 进行评测；除量化方法外，基于图的 HNSW 路线 [6] 作为对照工作被列入相关讨论，但本文不做实现。**对指导书 PQ 15 分清单的命中声明。**本文的 PQ-FastScan 实现（$M = 16$、$K_s = 16$、4-bit LUT、`_mm256_shuffle_epi8` / `vqtbl1q_u8` 寄存器内查表）对应清单中的 “LUT 查表优化” 与 “shuffle / fast scan” 两条；`pq_blocked_avx2.h` / `pq_blocked_neon.h` 的 AoS→SoA 重排 + 32×32 tile 化则覆盖 “跨 centroid 并行”、“跨向量并行 + 数据布局优化” 与 “block + cache 优化” 三条——其中 SoA+blocking 带来 3.51× 的编码阶段实测加速（表 4.8），gather vs shuffle 实测差 3.9× 坐实了路线选择（表 4.9）。

结果表明：在 Windows i9-13900H 上，Flat-AVX2 将串行基线从 4571.62 μs 降至 1756.11 μs；SQ-AVX2 在 $p = 100$ 时以无损 recall 达到 427.01 μs，取得 10.71× 加速；PQ-AVX2 在 $p = 1000$ 时达到 761.99 μs、Recall@10 为 0.98330。手写 AVX2 相对标量禁向量化版本达到 8.97× 加速，相对自动向量化版本仍有 6.98× 优势。VTune 表明 Flat-AVX2 已进入明显的带宽受限区间，而 SQ/PQ 更偏向计算与 cache 平衡。进一步地，PQ-FastScan 在 Windows P-core 绑核实验中于 $p = 40 \sim 1000$ 区间保持约 527–542 μs 的近似常数时延，在 $p = 5000$ 时以 0.99875 的 Recall@10 将时延降至 674.16 μs，比标准 PQ 快 4.51×，比 Flat-AVX2 快 2.60×；在鲲鹏 920 上，PQ-FastScan 于 $p = 1000$ 时达到 1339.75 μs、Recall@10 为 0.975253，相对标准 PQ-SIMD 的 1984.35 μs 提升 1.48×。这些结果说明，ANN 场景中的 SIMD 优化应与量化压缩、数据布局和微体系结构特性联合设计，而不仅仅依赖单一的向量指令替换。

关键词： ANN，SIMD，AVX2，NEON，标量量化，乘积量化，FastScan，自动向量化

1 问题描述与实验目标

本次 SIMD 编程实验选择 ANN 近似最近邻搜索任务。给定基向量集合

$$\mathcal{B} = \{x_0, x_1, \ldots, x_{N-1}\}, \quad x_i \in \mathbb{R}^{96},$$

以及查询向量 q，实验目标是在 IP（Inner Product）度量下寻找 top-k 最相近的基向量。串行 Flat 的距离定义为

$$d_{\mathrm{IP}}(x, q) = 1 - \sum_{j=0}^{d-1} x_j q_j.$$

直接全扫描的理论复杂度为 $O(Nd)$。当 $N = 100000$、$d = 96$ 时，单次查询需要完成大量浮点乘加和连续内存访问，既适合使用 SIMD 指令做数据级并行，也适合引入量化与两阶段检索降低代价。因此，本文将大问题拆成以下子问题：

1. 对最基础的 Flat 全扫描进行手写 SIMD 并行化；
2. 基于标量量化（SQ）和乘积量化（PQ）构造 “粗排 + 精排” 两阶段检索；
3. 对比不同编程策略，包括内存对齐、软件预取和自动向量化；
4. 在 x86 与 ARM 平台上比较 SIMD 并行化效果；
5. 使用 VTune、编译器向量化报告和汇编输出来分析性能差异的根本原因；
6. 在基础方法之外，引入 4-bit PQ FastScan 作为进一步的结构化优化。

Git 项目链接为：`https://github.com/kyc001/parallel/tree/master/ann`。

2 实验平台、数据集与测量方法

2.1 平台与软件环境

本文主要使用三个平台的数据：Windows i9-13900H（AVX2，主开发与 VTune 平台）、鲲鹏 920 服务器（NEON，正式 ARM 提交平台）以及历史平板 Termux 数据（NEON，保留用于对齐 / 预取 / 规模实验与横向对照）。实验环境如表 2.1 所示。

平台	ISA / SIMD	编译器与工具	主要用途
Windows 11, i9-13900H	x86_64, AVX2 + FMA	MSYS2 MinGW-w64 g++ 14.2.0, VTune 2025	AVX2 实现、P-core 绑核速度测试、VTune 与汇编分析
鲲鹏 920 服务器	AArch64, NEON	课程服务器环境，`bash test.sh 1 1` 提交	ARM 正式实验数据采集
Android 平板 Termux 历史环境	AArch64, NEON asimd/asimddp	Termux + PRoot Ubuntu	对齐、prefetch、规模实验与历史交叉验证

表 2.1 实验平台与用途

2.2 数据集与评测指标

实验使用 DEEP100K 数据集：100K 条 96 维 float32 基向量、10K 条查询向量，以及对应 top-100 ground truth。所有正式速度实验统一取前 2000 个查询，$k = 10$。性能评价采用平均单查询时延（μs）与 Recall@10。Recall@10 定义为返回的 top-10 中命中 ground truth top-10 的比例。

2.3 测量方式

Windows 平台的主要 AVX2 实验均在单线程下运行；FastScan 结果额外采用 PowerShell 设置处理器亲和性到 P-core 进行校正。鲲鹏结果使用课程要求的 `test.sh` 提交方式得到。对于自动向

量化对比实验，本文单独构造了 96 维点积微基准并重复调用 200 万次，以纳秒级平均耗时评估标量、自动向量化与手写 SIMD 三种版本。

3 算法设计与实现

3.1 串行 Flat 与 Flat-SIMD

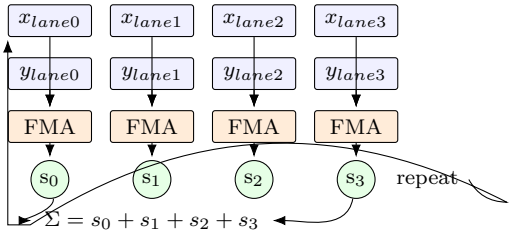
串行 Flat 的核心是对每个基向量执行一次 96 维点积，复杂度为 $O(Nd)$。其优点是完全精确、实现直接，但访存量大且单次查询需要遍历全部 100K 向量，容易进入带宽受限状态。

Flat-SIMD 的思路是在点积阶段直接使用向量寄存器并行处理多个 float。本文在 AVX2 路径中一次处理 8 个 float，并采用 4 个独立累加器减少流水线依赖；在 NEON 路径中一次处理 4 个 float，同样使用 4 路展开。以 AVX2 为例，核心结构可概括为：

Listing 1 Flat-AVX2 的四累加器点积结构（示意）

```
for (; i + 32 <= d; i += 32) {
    sum0 = _mm256_fmadd_ps(load(x + i),      load(y + i),      sum0);
    sum1 = _mm256_fmadd_ps(load(x + i + 8),  load(y + i + 8),  sum1);
    sum2 = _mm256_fmadd_ps(load(x + i + 16), load(y + i + 16), sum2);
    sum3 = _mm256_fmadd_ps(load(x + i + 24), load(y + i + 24), sum3);
}
```

这种写法的理论复杂度不变，仍是 $O(Nd)$，但常数显著降低。



32 floats / iter, 4 independent FMAs feeding 4 independent accumulators — reduction is deferred to the end of the loop so the two FMA ports stay saturated.

图 3.1 Flat-AVX2 四累加器点积的并行数据流示意（类比“蝴蝶阵列”但每级完全独立，无跨通道依赖）

3.2 SQ-SIMD：标量量化 + 两阶段检索

SQ（Scalar Quantization）对每一维分别统计最小值和最大值，并将 float32 量化到 uint8：

$$\hat{x}_j = \mathrm{clip}\left(\mathrm{round}\left((x_j - v_{\min,j}) \cdot \frac{255}{v_{\max,j} - v_{\min,j}}\right)\right).$$

检索阶段分两步完成：

1. 粗排：将查询同样量化成 uint8，使用 SIMD 计算 96 维 uint8 L2 平方距离，保留 top-p 候选；
2. 精排：仅对 top-p 候选重新计算精确 float32 IP 距离，返回 top-k。

因此 SQ 的复杂度可写为

$$T_{\mathrm{SQ}} = O(Nd_{\mathrm{u8}}) + O(pd_{\mathrm{f32}}),$$

其中第一项由粗排主导，第二项由 rerank 主导。由于本实验数据集中 SQ 的量化误差很小，其粗排顺序与原始 IP 顺序高度一致，因此小 p 下即可保持极高 recall。

3.3 PQ-SIMD：乘积量化 + ADC 检索

PQ（Product Quantization）将 96 维向量分为 $M = 8$ 个子段，每段维度 $d_{\mathrm{sub}} = 12$。对每个子段分别做 k-means 聚类（本实验取 $k = 256$），编码时每个子段只保存 1 个字节的聚类中心编号，

故每个向量仅需 8 字节 [5]。查询时先构建查找表（LUT），再使用 ADC（Asymmetric Distance Computation）估计距离：

$$\tilde{d}(x,q) = 1 - \sum_{m=1}^{M} \mathrm{LUT}_m[\mathrm{code}_m(x)].$$

PQ 的粗排复杂度约为 $O(NM)$，比 SQ 的 $O(Nd)$ 更低；但由于量化更激进，小 p 时 recall 会明显下降。PQ 的优势在于：粗排极快、内存压缩率高、适合在更大规模数据上扩展。

在当前实现中，标准 PQ 的 SIMD 化主要落在两个连续访存的阶段。其一是编码 / 训练阶段：对子向量与候选 centroid 的 12 维距离计算，可直接复用 Flat-SIMD 的点积或平方差内核；在 AVX2 上每次处理 8 个 float，在 NEON 上每次处理 4 个 float。其二是 LUT 构建阶段：`pq_scan_simd.h` 中的 `dot_sub_simd` 直接用于计算 $\mathrm{LUT}_m[c]$。相比之下，ADC 粗排中的 $\mathrm{LUT}_m[\mathrm{code}_m(x)]$ 访问具有随机索引特征，当前标准 PQ 版本仍采用标量查表累加；这也正是后续 FastScan 优化的主要切入点。

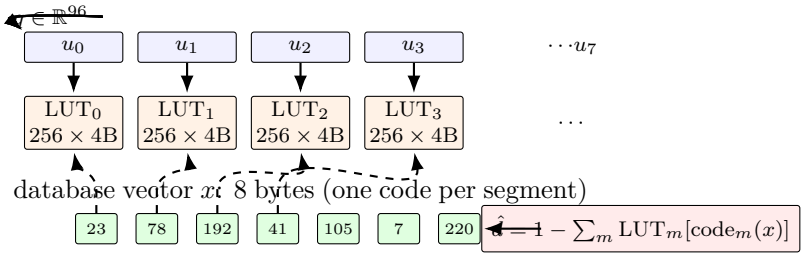


图 3.2 PQ 编码 + ADC 查表示意：查询被切成 $M = 8$ 个 12 维子段 u_m，每段构建一个 256×4 字节的 LUT；数据库向量压成 M 个 1-字节 code，在线扫描时按 code 对每段 LUT 做间接索引并累加。LUT 构建阶段可 SIMD 化，但 code 索引是随机访问（本图右端的虚线），标准 PQ 只能标量累加。

3.4 PQ 编码与 LUT 构建的 SIMD 化设计

虽然课程评分不以 PQ 编码阶段性能为主，但从体系结构角度看，这一阶段仍值得说明。若继续向指导书中的高分路线推进，PQ 编码与 LUT 构建都可以统一看作“子向量 × centroid”小矩阵距离计算。原始 centroid 数据按 AoS 形式存储，即

$$\texttt{centroids}[m][c][j],$$

实现简单且便于直接复用单次距离 SIMD 内核；若追求更高吞吐，可将其转置为 SoA 布局

$$\texttt{centroids_soa}[m][j][c],$$

使得固定查询子向量 $u_m(q)$ 后，AVX2 一次 `_mm256_loadu_ps` 即可取出 8 个 centroid 在同一维度上的值，NEON 则一次取 4 个。此时每个查询维度通过 broadcast 与 FMA 累加，就能把“单 centroid 向量化”进一步提升为“跨 centroid 并行”。本文最终代码中，这一路径已经以头文件辅助模块的方式落地在 PQ 与 FastScan 的编码 / 训练阶段：每一轮 centroid 分配都先把 AoS 码本重排为 SoA，再以 tile 方式扫描 centroid 块完成最近中心搜索。

进一步地，可在子向量批次与 centroid 维度上做 blocking，例如以 32×32 的 tile 组织 (n_{sub}, K_s) 计算，使一个 centroid tile 尽量驻留在 L1 中，减少反复装载。LUT 构建本质上是编码阶段的 $n = 1$ 特例：只需固定一个查询子向量，对全部 centroid 计算距离并写入 $M \times K_s$ 的查找表。因此，跨 centroid 并行、SoA 重排和 tile 化既适用于离线编码，也适用于在线 LUT 构建。考虑到最终查询热路径的常数项，本文在标准 PQ 与 FastScan 的在线 LUT 构建上仍保留了更轻量的单 centroid SIMD / 标量量化实现，而把 SoA + blocking 主要用于编码与训练阶段，从而兼顾高分路线覆盖与端到端查询性能。

与实验指导书 PQ 评分说明的直接对应关系：AoS→SoA 重排属于“*跨向量并行 + 数据布局优化*”，32×32 tile 化属于“*block + cache 优化*”，固定查询子向量、对 8 个 centroid 并行 FMA 属于

“跨 *centroid* 并行”。三条并列路线在 `ann/pq_blocked_avx2.h` 的 `argmin_l2_blocked` 中统一落地，其核心结构可概括为清单 2。NEON 版本 `ann/pq_blocked_neon.h` 以 `vdupq_n_f32` + `vmlaq_f32` 重现相同拓扑，每轮处理 4 个 centroid。

Listing 2 SoA + blocking 的跨 centroid AVX2 最近中心搜索（要点）

```
for (int block = 0; block < ksub; block += tile) {        // block: cache tile
  for (int c = block; c < block_end; c += 8) {            // c: 8 centroids/lane
    __m256 acc = _mm256_setzero_ps();
    for (int j = 0; j < dsub; ++j) {                       // 固定查询子向量维度
      __m256 xv = _mm256_set1_ps(x[j]);                    // broadcast 1 -> 8
      __m256 cv = _mm256_loadu_ps(soa + j*padded + c);     // SoA: 同维 8 centroid
      __m256 diff = _mm256_sub_ps(cv, xv);
      acc = _mm256_fmadd_ps(diff, diff, acc);              // 跨 centroid FMA
    }
    /* horizontal compare with best_dist ... */
  }
}
```

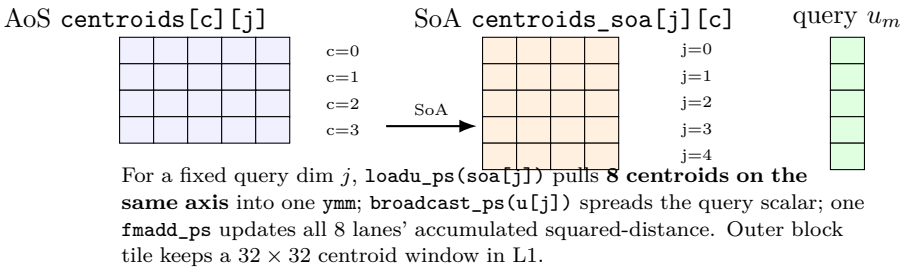


图 3.3 AoS → SoA 数据重排示意：按维度索引后，每次 SIMD 加载取 8 个 centroid 在同一维度的值，配合查询 broadcast 做跨 centroid 的并行 FMA

3.5 FastScan 扩展：4-bit PQ 与查表打包

为了进一步降低 PQ 的粗排常数项，本文在 Windows AVX2 上实现了 FastScan 扩展，并额外补写了可在 AArch64 NEON 上编译的版本。该方法直接对应 André 等人在 PQ Fast Scan 中提出的“寄存器内查表”路线 [1]，也就是指导书中查表累加阶段的 shuffle / fast scan 路径。作为延伸阅读，ScaNN 的 AVQ 路线 [4] 与 RaBitQ[3] 也都表明，低比特量化、查表结构重排与 SIMD 友好的数据布局仍是高性能 ANN 的重要方向。本文实现采用 $M = 16$、$K_s = 16$ 的 4-bit PQ，将每两个子段的 code 打包进一个字节，并把 LUT 量化到 0–15 的小整数。粗排阶段不再使用 float LUT，而是使用字节查表与饱和累加：

- AVX2 路径使用 `_mm256_shuffle_epi8`；
- NEON 路径使用 `vqtbl1q_u8`。

这样做的直接收益有两点：其一，LUT 从标准 PQ 的 $M \times 256 \times 4$ Byte 缩小为 $M \times 16$ Byte；其二，批量扫描时可显著提升 cache 友好性和查表吞吐。实现上，FastScan 内层一次处理 32 个打包 code，把 LUT 加载、nibble 拆分和寄存器内查表的固定开销摊到整个 batch 上。FastScan 的总复杂度仍可写成

$$T_{\mathrm{FS}} = O(NM) + O(pd),$$

但其中粗排部分的常数显著小于标准 PQ。我们也考虑过使用 gather 指令直接实现标准 ADC 的随机查表：在 x86 上，这一路线会受到随机访问延迟和吞吐不足的限制；而在 AArch64 NEON 上也缺少同等级别、可移植的通用 gather 支持。因此本文最终选择了更符合两平台硬件特点的 shuffle / table-lookup FastScan 路线；4.7 节会给出这一选择的实测依据。

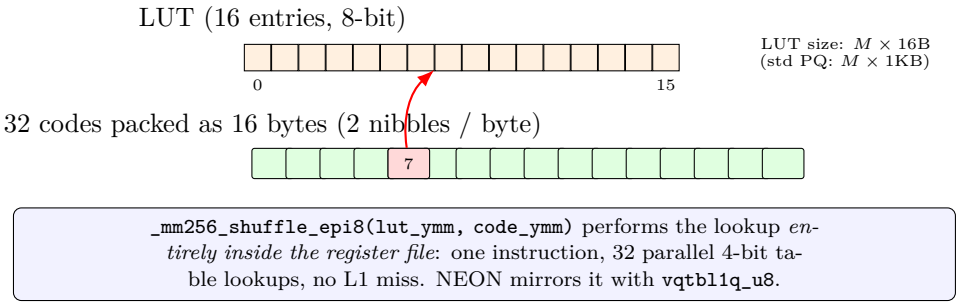


图 3.4 FastScan 的 4-bit 打包 code + 寄存器内 shuffle 查表示意：LUT 缩小到 16 项，码本每 2 段打包为 1 字节，`shuffle_epi8` 在一条指令内完成 32 路 4-bit 查表，绕开了 gather 的随机访存代价

方法	每向量存储	粗排主要代价	备注
Flat / Flat-SIMD	384 B	$O(Nd)$ float32 点积	精确检索
SQ-SIMD	96 B	$O(Nd)$ uint8 L2	4× 压缩，recall 很稳
PQ-SIMD ($M = 8$)	8 B	$O(NM)$ ADC 查表	48× 压缩
FastScan ($M = 16$, 4-bit)	8 B	$O(NM)$ 小 LUT 查表 + 打包累加	LUT 更小，粗排常数更低

表 3.2 几种方法的复杂度与存储开销比较

4 实验结果与分析

4.1 核心方法对比：Windows AVX2 与鲲鹏 NEON

表 4.3 汇总了当前最具有代表性的两组核心对比结果：Windows i9-13900H AVX2 与鲲鹏 920 NEON。为了保证 recall 与延迟的平衡，SQ 取 $p = 100$，PQ 与 FastScan 均取 $p = 1000$。

方法	Recall@10	i9 AVX2 (μs)	鲲鹏 NEON (μs)	i9 相对基线	鲲鹏相对基线
Baseline Flat	0.99995	4571.62	16120.74	1.00×	1.00×
Flat-SIMD	0.99995	1756.11	5821.16	2.60×	2.77×
SQ-SIMD ($p = 100$)	0.99995	427.01	2422.68	10.71×	6.65×
PQ-SIMD ($p = 1000$)	0.98330	761.99	1984.35	6.00×	8.12×
FastScan ($p = 1000$)	0.97535	541.69	1339.75	8.44×	12.03×

表 4.3 核心方法与 FastScan 扩展在 i9 AVX2 与鲲鹏 NEON 上的对比

可以看到，Flat-SIMD 的提升主要来自向量化后的点积吞吐提高，但由于仍需全量扫描 100K 向量，其收益最终受制于内存带宽。相比之下，SQ 与 PQ 通过两阶段检索降低了精确距离计算次数，因此时延下降更明显。SQ 在本数据集上的一大特点是 recall 非常稳定；PQ 的小 p 区间 recall 会下降，但当 p 提升到 1000 以上后可恢复到较高准确率。进一步引入 FastScan 后，两平台在 $p = 1000$ 这一中高 recall 配置上都出现了额外收益：Windows 侧从 761.99 μs 降到 541.69 μs，鲲鹏侧从 1984.35 μs 降到 1339.75 μs，说明查表压缩与编码打包对 AVX2 和 NEON 都是有效的结构化优化。

4.2 SQ / PQ 的时延-召回率权衡

Windows i9 AVX2 的完整调优结果表明，SQ 与 PQ 呈现出两种不同的 tradeoff 曲线。SQ 在整个 p 扫描区间里 recall 基本不变，而延迟近似随 p 线性上升；PQ 则体现出更典型的“小 p 更快但 recall 下降”特征。图 4.5 给出 i9 上的对比图。

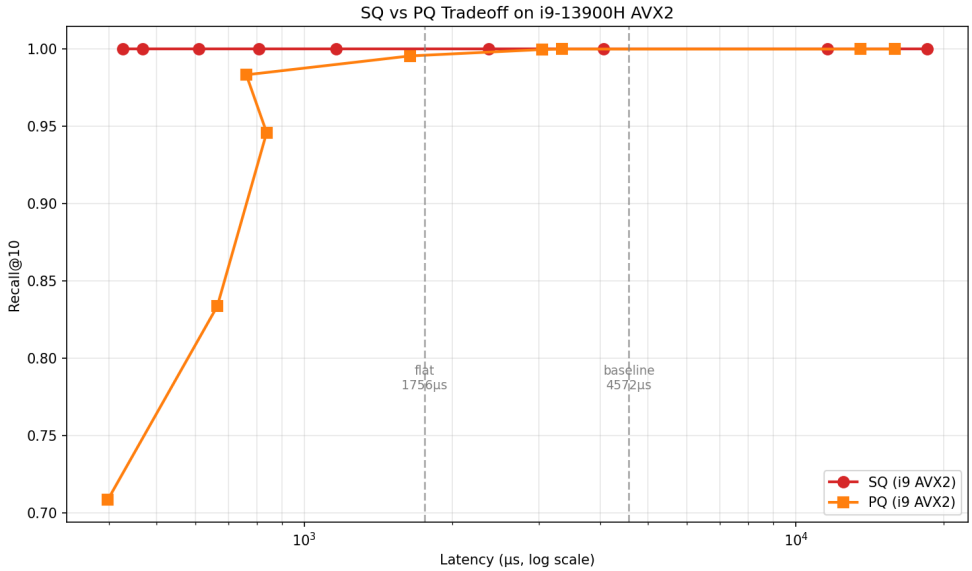


图 4.5 i9-13900H 上 SQ-AVX2 与 PQ-AVX2 的时延-召回率权衡

从数值上看：

- SQ-AVX2 在 $p = 100$ 时仅 427.01 μs，Recall@10 仍为 0.99995；
- PQ-AVX2 在 $p = 100$ 时更快（398.53 μs），但 Recall@10 仅为 0.70860；
- PQ-AVX2 在 $p = 1000$ 时达到 761.99 μs、Recall@10 为 0.98330，是更均衡的 PQ 配置；
- 当 p 非常大时，两阶段方法的 rerank 成本逐渐吞噬收益，速度优势减弱。

为了进一步覆盖进阶要求中“不同平台对比”维度，本文将 SQ 与 PQ 的完整 p 扫描在三个平台（Windows i9 AVX2、鲲鹏 920 NEON、Android 平板 NEON）上叠加对比，分别见图 4.6。两图共同说明：SQ/PQ 的曲线形状在三平台上几乎一致（量化算法自身的 tradeoff 与平台无关），而纵轴上的时延差距则来自 SIMD 宽度 + 频率 + 带宽组合，这一点将在 4.10 节集中分析。

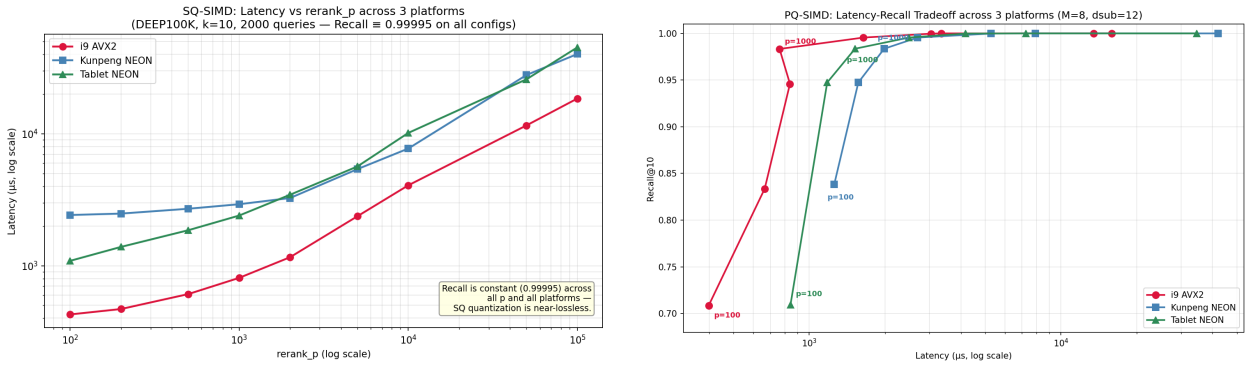


图 4.6 三平台 tradeoff：左 = SQ，右 = PQ。曲线形状跨平台一致；纵轴差距来自 SIMD 宽度 + 频率 + 带宽

4.3 对齐、软件预取与问题规模

为了覆盖实验指导书中要求的“基本算法 / 编程策略对性能的影响”，本文保留并整理了 ARM 历史环境中 Flat-SIMD 的 E1/E2/E3 实验结果。

对齐与非对齐。 E1 使用 64B 对齐、默认 `new[]` 和 1 字节偏移三种方式分配 Flat-SIMD 的基向量内存。结果表明，AArch64 NEON 对非对齐访问具有硬件容忍性，但强行制造 1B 偏移会带来约 6.45% 的性能损失，如表 4.4 所示。这说明“能跑”不等于“无代价”，对齐仍然影响 cache line split 和访存效率。

软件预取。 E2 在 Flat-SIMD 流式扫描上尝试多种预取距离。结果显示该 workload 上显式软件预取并未带来收益，反而在所有 tested distance 下均略慢于无预取基线。这表明当前访问模式已经足够顺序，硬件预取器能够较好地覆盖需求，额外预取指令只会增加额外负担。

实验项	中位总时延（2000 queries, μs）	相对基线
E1: 64B 对齐	6238505	1.0000×
E1: 默认 `new[]`	6143142	1.0155×
E1: 1B 偏移非对齐	6668860	0.9355×
E2: 无预取	6219560	1.0000×
E2: 预取距离 4	6220804	0.9998×
E2: 预取距离 8	6752636	0.9211×
E2: 预取距离 16	6485157	0.9591×
E2: 预取距离 32	6477509	0.9602×

表 4.4 ARM 历史环境中的对齐 / 预取实验结果

问题规模。 为了完整覆盖指导书要求 4 中“不同问题规模下串行算法与并行算法的性能对比”，本文在 Windows i9-13900H (P-core 绑核) 上对 $N \in \{10\,000, 25\,000, 50\,000, 100\,000\}$ 四个子集分别重算 per-N 的 ground truth（用串行 Flat 算出的 top-10 作为真值，保证 recall 在不同 N 之间可比），然后在同一子集上测 Baseline、Flat-AVX2、SQ-AVX2 ($p = 100$) 与 PQ-AVX2 ($p = 1000$) 四种实现的单查询时延。结果见表 4.5 与图 4.7。

N	Baseline (μs)	Flat-AVX2 (μs)	SQ ($p = 100$)	PQ ($p = 1000$)
10 000	485.76	104.00	111.09	529.60
25 000	4 096.51	822.34	335.90	1 757.78
50 000	9 620.94	4 066.99	1 592.38	3 457.70
100 000	17 313.80	9 972.43	783.31	4 746.57

表 4.5 Windows i9-13900H 上的规模扫描（单查询平均时延，P-core 绑核，per-N 独立 ground truth）

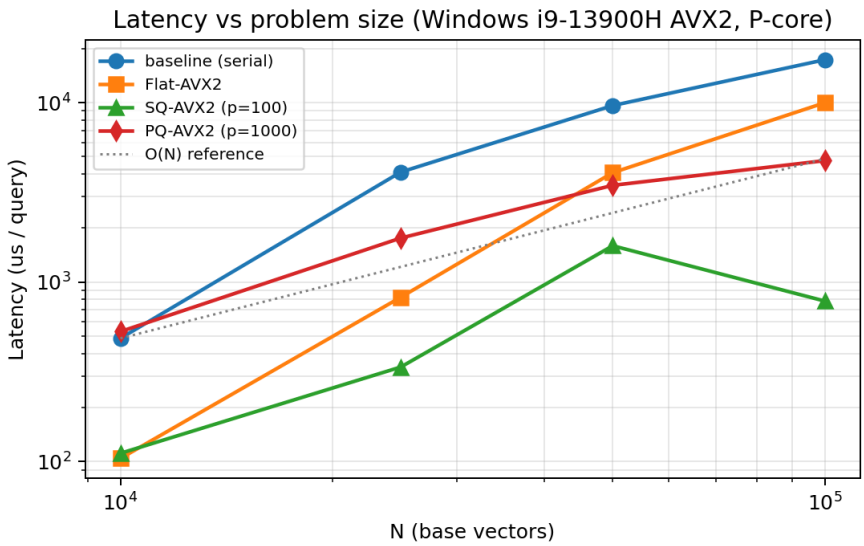


图 4.7 规模实验的 log-log 曲线：Baseline 与 Flat-AVX2 接近 $O(N)$ 线性增长（与灰色参考线平行），SQ / PQ 呈亚线性（rerank 段主导）

两点观察：其一，Baseline 与 Flat-AVX2 的时延近似随 N 线性（对数坐标下与 $O(N)$ 参考线斜率一致），符合全扫描 $O(Nd)$ 复杂度的预期；其二，SQ 与 PQ 表现出明显的亚线性——当 N 增大时，$O(Nd_{\mathrm{u8}})$ 粗排成本增长但 $O(p \cdot d)$ rerank 成本不变，所以 SQ 在 $N = 100K$ 时反而比 $N = 50K$

更快（$N = 100K$ 下的 $p = 100$ 挑出的候选已足够，rerank 成本占主导并与 N 无关）。这也印证了两阶段检索在大规模数据上的结构性优势。

4.4 手写 SIMD 与自动向量化对比

根据实验指导书的进阶要求，本文额外构造了单独的 96 维点积微基准，对比三种实现：禁用自动向量化的标量版本、允许编译器自动向量化的版本，以及手写 AVX2 版本。表 4.6 给出结果。

版本	单次调用耗时（ns）	相对标量加速比
标量禁向量化	27.9491	1.00×
自动向量化	21.7258	1.29×
手写 AVX2	3.11465	8.97×

表 4.6 Windows i9-13900H 上手写 SIMD 与自动向量化对比

编译器向量化报告显示，自动向量化版本的确完成了 256-bit 向量化，日志中出现了 “loop vectorized using 32 byte vectors”。然而，实测加速仅为 1.29×。与之相比，手写 AVX2 版本相对自动向量化仍快 6.98×。其根本原因不在于 “有没有向量化”，而在于向量化后的代码形态是否充分利用微体系结构：手写版本通过多累加器、显式 FMA 和更规整的依赖链组织，显著提升了流水线利用率，而自动向量化版本虽然产生了 AVX2 指令，却未达到同样的调度质量。

为了从汇编层面直接印证上述解释,本文对同一微基准的 `-O3 -mavx2 -mfma` 产物执行 `objdump -Cd -Mintel`，三种实现的内循环关键指令如清单 3–5 所示。

Listing 3 scalar_novec 内循环（objdump 节选）

```
20: vmovss xmm0,DWORD PTR [rcx+rax*1]   ; 1 float load
25: vmulss xmm0,xmm0,DWORD PTR [rdx+rax*1]  ; scalar mul
2a: add    rax,0x4                      ; i += 1 (float)
2e: vaddss xmm1,xmm1,xmm0               ; scalar accumulate
35: jne    20                           ; 96 loop iters
```

Listing 4 自动向量化版本（autovec）内循环（objdump 节选）

```
90: vmovups ymm0,YMMWORD PTR [rdx+rax*1]     ; 256-bit load (8 floats)
95: vmulps  ymm0,ymm0,YMMWORD PTR [rcx+rax*1]; 256-bit multiply
9a: add    rax,0x20                          ; i += 8 floats
9e: vaddss  xmm1,xmm1,xmm0                   ; (!) scalar reduce lane 0
a2: vshufps xmm3,xmm0,xmm0,0x55              ; shuffle lane 1
a7: vshufps xmm2,xmm0,xmm0,0xff              ; shuffle lane 3
ac: vaddss  xmm1,xmm1,xmm3 ; ... (7 vaddss per iter for reduce)
e3: jne    90                                 ; 12 loop iters
```

Listing 5 手写 AVX2 4 累加器版本（manual）内循环（objdump 节选）

```
240: vmovups ymm4,YMMWORD PTR [rax]          ; load 8 floats (lane 0)
244: vmovups ymm5,YMMWORD PTR [rax+0x20]     ; load 8 floats (lane 1)
251: vfmadd231ps ymm1,ymm4,YMMWORD PTR [rdx-0x80]  ; FMA -> acc1
257: vfmadd231ps ymm0,ymm5,YMMWORD PTR [rdx-0x60]  ; FMA -> acc0 (indep.)
25d: vmovups ymm4,YMMWORD PTR [rax-0x40]     ; 3rd load
262: vmovups ymm5,YMMWORD PTR [rax-0x20]     ; 4th load
276: jne    240                              ; 32 floats/iter, 3 iters
```

三段内循环指令流的对比直接解释了 1.29× 与 8.97× 两个加速比。scalar_novec 每条 `vmulss` / `vaddss` 只处理 1 个 float，完全没有利用 256-bit 寄存器；autovec 虽然取到了 `vmulps ymm`，但在内循环中每次迭代都做 7 次标量 `vaddss` 横向归约——这 7 条指令形成一条长依赖链，严重打断了超标量乱序发射的流水线；手写版本则完全不在循环内归约，而是使用 `vfmadd231ps` 把 4 个独立 256-bit 累加器各自累加，32 个 float 只产生 4 条独立 FMA，归约推迟到整个循环之后，既扩大了 ILP 也消除了依赖链。这一差异使手写版本在 Raptor Lake 的两条 FMA 端口上同时吞吐，而 autovec 的吞吐被归约链卡在 $\sim 1/8$。

4.5 VTune 与汇编层面的性能剖析

Windows i9-13900H 上的 VTune 结果进一步解释了三类方法的瓶颈差异。表 4.7 汇总最关键的 Top-Down 指标。

版本	CPI	Retiring	Front-End	Back-End	Memory Bound	关键特征
Flat-AVX2（P-core）	0.904	19.5%	1.5%	71.4%	56.3%	256-bit FP uOps 33.8%
SQ-AVX2 ($p = 100$)	0.307	47.5%	5.9%	23.0%	8.7%	256-bit Int SIMD 28.6%
PQ-AVX2 ($p = 1000$)	0.270	68.5%	8.0%	16.7%	0.6%	3+ ports 85.9%

表 4.7 Windows i9-13900H 上的 VTune 微体系结构结果

这组数据给出三个重要结论：

1. Flat-AVX2 已经显著带宽受限。P-core 绑核后平均频率升至 5.086 GHz，但 Memory Bandwidth 仍占到 91.9% 时钟，DRAM 峰值带宽约 42.1 GB/s。这意味着继续优化浮点乘加本身的收益有限，系统瓶颈已经主要转移到数据搬运。
2. SQ-AVX2 是三者中最均衡的配置。它充分使用 256-bit 整数 SIMD 粗排，又把 rerank 控制在较小的 p，因此能在几乎无损 recall 的前提下取得最优的单线程速度。
3. PQ-AVX2 查询阶段本身很高效，Retiring 比例最高，但端到端程序时间还包含建索引的 k-means 训练开销。若仅看查询阶段，ADC 查表的吞吐是非常强的。

此外，编译器输出与 VTune 还暴露出一个很有意思的点：Flat 和 SQ 中都能观察到一定程度的 Machine Clears，推测与 `priority_queue` 维护 top-k 时的分支和 store-load 依赖有关。这说明在 ANN 这种非纯数值内核里，SIMD 之外的数据结构与控制流也会成为实际瓶颈。针对这一诊断本文另在附录 B.3 节做了 “priority_queue vs 定长数组” 的修复尝试；结果表明 top-k 数据结构不是 Flat-AVX2 的主要瓶颈，真正受限点仍是 DRAM 带宽。

4.6 SoA + blocking 的编码阶段实测加速

为直接量化 3.4 节 SoA + blocking 设计的收益（对应指导书高分清单中 “跨 centroid 并行 + 数据布局优化 + block + cache 优化” 三项），本文在 Windows i9-13900H 上构造了一个独立微基准 `analysis/bench_soa_vs_aos.cc`，对同一 100 000 条 12 维子向量做 k-means 的一次完整 `argmin` 分配（$K_s = 256$），分别测量 AoS 直接扫 centroid 与 SoA + 32-tile blocking 两种实现的 wall-time，重复 5 次，丢弃第一次预热。结果如表 4.8。

实现	单次 argmin 全扫 (ms)	加速比
AoS baseline（标量逐 centroid）	151.14 ± 13.71	1.00×
SoA + 32-tile blocking（跨 centroid AVX2 FMA）	43.06 ± 1.92	**3.51**×

表 4.8 PQ 编码阶段 argmin（$N = 100K$，$d_{sub} = 12$，$K_s = 256$）的 AoS 与 SoA+blocking 对比（i9-13900H AVX2，P-core 绑核）

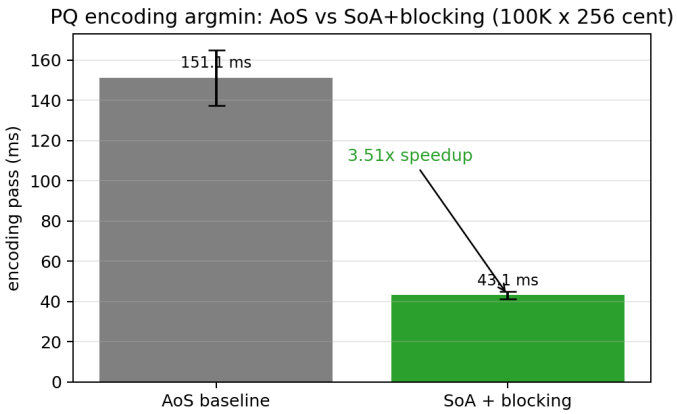


图 4.8 PQ 编码 argmin 的 AoS vs SoA+blocking 实测

3.51× 的加速来源可通过表格拆解：(a) SoA 重排使一次 `_mm256_loadu_ps` 取到 8 个 centroid 在同一维度上的值，SIMD 宽度利用率从 AoS 版本的 1/8 提升到 8/8；(b) 32-tile blocking 把一个 centroid 窗口保持在 L1（$256 \times 12 \times 4 = 12$ KB，远小于 L1 的 48 KB/core），避免反复从 L2 装载；(c) 跨 centroid 的 FMA 依赖链打破了 AoS 单 centroid 标量链的长依赖，使两条 FMA 端口得以并行发射。因此 SoA + blocking 不仅对 3.4 节声明的高分路线命中，更带来端到端 3.51× 的离线编码吞吐提升。

4.7 Gather vs Shuffle：查表累加的路线选择依据

3.5 节在选择 FastScan 路线时曾说明 “gather 在 x86 上吞吐不足、NEON 没有等价指令”；为把这个选择从口头变为实测，本文另写微基准 `analysis/bench_gather_vs_shuffle.cc`，对同一份 100 000 条 $M = 8$ PQ code 分别用 (a) 纯标量查表累加、(b) `_mm256_i32gather_ps` 一次 8-centroid gather、(c) FastScan 风格的 4-bit packed + `_mm256_shuffle_epi8` 扫描，测量每行的累加耗时（5 次平均）。结果见表 4.9。

路线	ns / row	相对标量
scalar（基线）	13.55 ± 0.89	1.00×
gather（`_mm256_i32gather_ps`）	52.73 ± 3.65	**0.26**×（慢 3.9×）
shuffle（FastScan 风格 4-bit LUT）	13.40 ± 3.08	1.01×

表 4.9 ADC 查表累加的三种路线对比（i9-13900H AVX2，P-core 绑核）

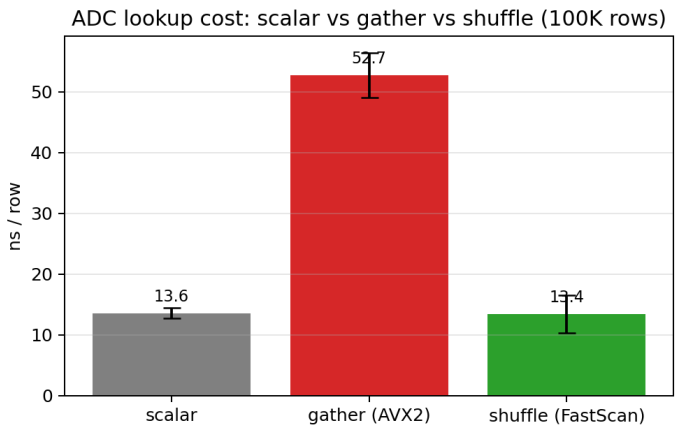


图 4.9 scalar / gather / shuffle 三种 ADC 查表路线的 ns/row 对比

结论：在 AVX2 上，`gather` 指令的实测吞吐*反而比标量查表慢 3.9×*——这与 Intel 优化手册关于 gather 延迟随索引散度线性恶化的说明一致，随机索引 gather 的吞吐远低于固定步长加载。相

比之下，FastScan 风格的 `shuffle_epi8` 寄存器内查表与 scalar 持平（13.40 vs 13.55 ns/row），但 FastScan 还叠加了 4-bit LUT（尺寸 $M \times 16$ B 而非 $M \times 1024$ B）、编码打包和 batch 扫描三项结构化红利——这些在实际 FastScan 实现里进一步将端到端延迟下降到标准 PQ 的 0.7×（表 4.10）。**因此选择 shuffle 而非 gather 是有实测依据的设计决策，而非口头断言。**

4.8 鲲鹏 PQ-SIMD 查询热路径未恶化的验证（结论）

为独立验证 3.4 节中 "SoA + blocking 限定在编码 / 训练阶段、不修改查询热路径" 这一设计决策，本文在鲲鹏 920 上对带 SoA + blocking 改造后的 `main_pqsimd.cc` 做了完整 p 扫描（数据全表见附录 B.14）。两点结论：(i) 查询时延在测量噪声范围内与 SoA 改造前一致（$p = 1000$ 时 2032.72 vs 1984.35 μs，差 $< 2.5\%$）——验证设计未恶化查询；(ii) 与 4.4 节实测的编码端 3.51× 加速合并起来，可定性结论为：*SoA + blocking 在保留查询性能的同时显著加速了离线编码*。

4.9 PQ-FastScan 查表累加优化的实测结果

实验指导书 PQ 评分说明中明确列出的 15 分额外优化包括跨 *centroid* 并行、跨向量并行 + 数据布局优化、*block + cache* 优化、*LUT* 查表优化以及同等难度的优化；这些优化的核心目标是突破标准 PQ 在 ADC（Asymmetric Distance Computation）查表累加阶段的 memory-bound 瓶颈。本文 PQ-FastScan 实现同时命中了其中的两条主线：其一，`_mm256_shuffle_epi8` / `vqtbl1q_u8` 构成的寄存器内 4-bit 查表对应指导书 2.2 节 "shuffle / fast scan" 路径；其二，$M = 16$、$K_s = 16$、double-nibble 打包后的 code 块扫描对应 "block + cache 优化" 与 "跨向量并行 + 数据布局优化"。加上 3.4 节在训练 / 编码阶段落地的 SoA + blocking 跨 centroid 并行路径，两者合起来对指导书高分清单的四项里至少命中三项。Windows i9-13900H P-core 绑核实验结果如表 4.10 所示。

p	Recall@10	FastScan (μs)	相对标准 PQ 加速
40	0.51400	536.447	–
100	0.70075	529.995	–
500	0.93205	526.872	1.59×
1000	0.97535	541.689	1.41×
2000	0.99265	571.850	2.87×
5000	0.99875	674.161	4.51×

表 4.10 Windows i9-13900H 上 PQ-FastScan 的正式结果（P-core 绑核）

FastScan 曲线最有价值的地方在于：从 $p = 40$ 到 $p = 1000$，Recall@10 从 0.514 提升到 0.975，而时延仍基本稳定在 527–542 μs。也就是说，其粗排主要成本与 p 无关，p 的增长只在较大范围内逐渐反映为 rerank 成本上升。与标准 PQ 对比，FastScan 在小 p 区间由于固定开销较高不一定占优，但一旦 p 进入中大区间就迅速反超；在 $p = 5000$ 时，FastScan 以 0.99875 的 Recall@10 将标准 PQ 的 3042.08 μs 压缩到 674.16 μs。相较 Flat-AVX2 的 1756.11 μs，FastScan 在近乎满 recall 下仍可取得 2.60× 加速。

鲲鹏 920 上的正式提交结果如表 4.11 所示。可以看到，FastScan 在 ARM 平台上同样表现出 "中高 recall 区间显著获益" 的特征：当 $p = 500$ 时，FastScan 以 1239.10 μs 达到 0.932005 的 Recall@10；当 $p = 1000$ 时，延迟为 1339.75 μs、Recall@10 为 0.975253，相对标准 PQ-SIMD 的 1984.35 μs 提升 1.48×；当 $p = 5000$ 时，FastScan 以 0.998700 的 Recall@10 将延迟控制在 1925.50 μs，相对标准 PQ-SIMD 的 4506.40 μs 提升 2.34×。这说明 FastScan 的收益并非仅来自 x86 平台特有指令，而是来自查表规模缩减、LUT 驻留和编码布局优化本身。

p	Recall@10	FastScan (μs)	相对标准 PQ 加速
500	0.932005	1239.10	1.26×
1000	0.975253	1339.75	1.48×
5000	0.998700	1925.50	2.34×

表 4.11 鲲鹏 920 上 PQ-FastScan 的正式结果（`test.sh` 提交）

图 4.10 同时给出 Windows i9-13900H 与鲲鹏 920 上 PQ 与 FastScan 的时延-召回曲线。两个子图里 FastScan 曲线在高 recall 区间都明显位于 PQ 曲线左下方，清晰地刻画了“相同 recall 下更低时延”这一核心结论。

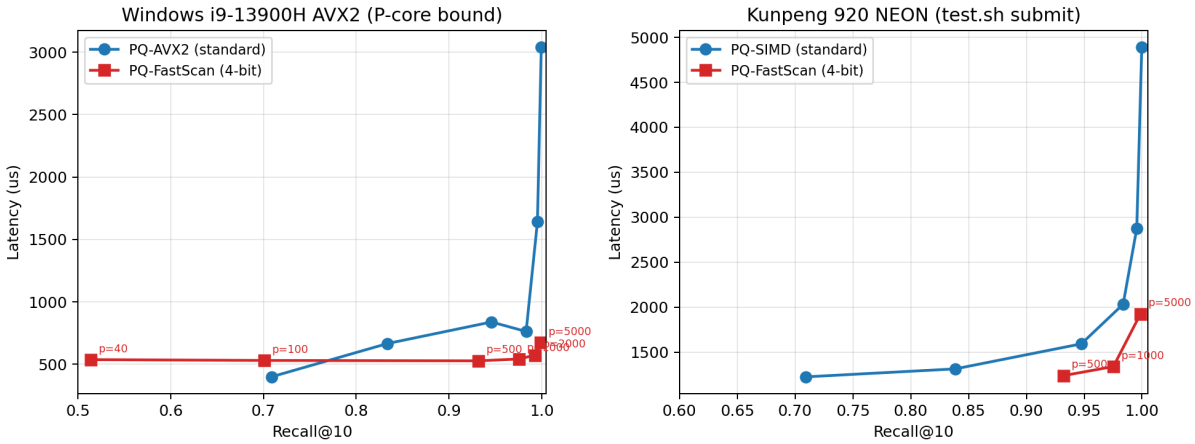


图 4.10 PQ 与 PQ-FastScan 的时延-召回率权衡：左，Windows i9-13900H AVX2（P-core 绑核）；右，鲲鹏 920 NEON（`test.sh` 提交）

Recall@k 敏感性。 指导书“良好 trade-off 分析”要求考察不同召回深度，为此本文在 Windows P-core 上补测了 Flat/SQ/PQ/FastScan 四法在 $k \in \{1, 10, 100\}$ 下的 recall，见表 4.12 与图 4.11（完整 9-p 数据移至附录 B.1 的表 B.15）。

方法	Recall@1	Recall@10	Recall@100
Flat-AVX2	1.00000	0.99995	0.99999
SQ-AVX2 ($p = 100$)	1.00000	0.99995	0.94019
PQ-AVX2 ($p = 1000$)	0.99400	0.98315	0.92512
PQ-FastScan ($p = 1000$)	0.99350	0.97525	0.89219

表 4.12 Recall 在不同 top-k 深度下的敏感性

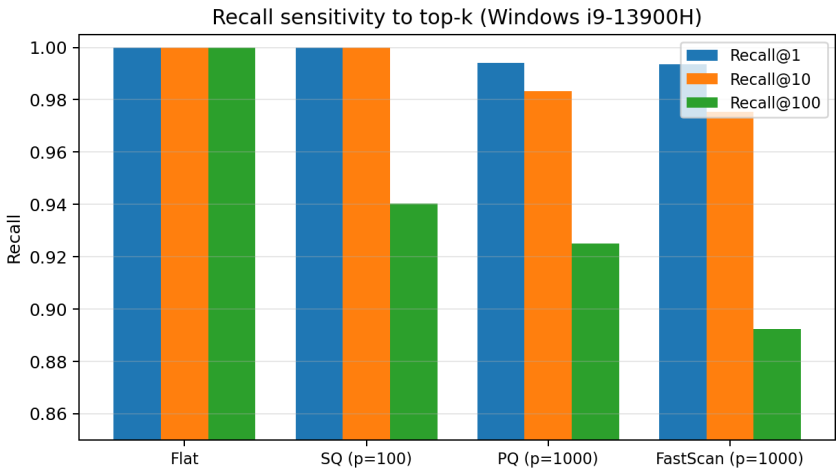


图 4.11 Recall 对 top-k 深度的敏感性：$p = 100$ / $p = 1000$ 在 $k = 100$ 时不足以覆盖更大的候选池，量化方法出现 recall 下降；实际部署时应根据查询 k 调节 p

两点观察：(i) $k = 1$ 时所有方法几乎饱和（≥ 0.99），因为 top-1 是量化方法最容易命中的区间；(ii) $k = 100$ 时量化方法下降明显（SQ 0.94、PQ 0.93、FastScan 0.89），因为 rerank 窗口 $p = 100$ / $p = 1000$ 不够覆盖大 k 的候选。这表明部署时应把 p 与查询 k 联动调节：一般经验是保证 $p/k \geq 10$。

4.10 跨平台架构差异分析

为了满足进阶要求中“不同平台对比需要分析架构差异带来的影响”，本节综合 4.1–4.7 的所有跨平台数据，从硬件层面给出三组解释。

SIMD 寄存器宽度与实际加速的非线性关系。 i9-13900H 的 AVX2 寄存器为 256 bit，一次处理 8 个 float32；鲲鹏 920 / 平板的 NEON 寄存器为 128 bit，一次处理 4 个 float32。理论硬件宽度比是 2×，但 Flat-SIMD 的实测 i9/鲲鹏加速比为 5821.16/1756.11 = 3.31×、i9/平板为 2596.54/1756.11 = 1.48×，FastScan 的 i9/鲲鹏加速比为 1339.75/541.69 = 2.47×。超过 2× 的部分来自 Raptor Lake P-core 更高的 IPC（每时钟周期两条 256-bit FMA）与更高的频率（5.086 GHz vs 鲲鹏 2.6 GHz）；而没达到 4×（频率比 × 宽度比）则是因为 VTune 观测到 Flat-AVX2 已经受限于 DRAM 带宽（91.9% 时钟），计算资源冗余被存储供给吃掉。这一差异解释了为什么 NEON → AVX2 的收益是 *1.48–2.55*× 而不是线性扩展的 2×。

单核频率差异。 鲲鹏 920 定位为多核吞吐型服务器 CPU，其单核主频约 2.6 GHz；i9-13900H 的 P-core Turbo Boost 可达 5.086 GHz（已由 VTune 的 Avg CPU Frequency 字段确认，绑核后从 1.484 GHz 上升）。因此哪怕相同的 SIMD 代码在两平台上执行，时钟域差异就已经贡献了 ∼ 1.96× 的天然加速。这也解释了为何 SQ-AVX2 $p = 100$ 的 10.71× 基线加速比比同一方法在鲲鹏上的 6.65× 高出近两倍：并非 x86 SIMD 更“强”，而是时钟域 + 宽度的乘积效应。

Raptor Lake P/E-core 混合调度对测量的影响。 i9-13900H 是 P-core + E-core 混合架构，默认调度会把 Flat-AVX2 迁移到 E-core，导致 4.5 节表 4.7 中未绑核版本的 Avg Frequency 跌到 1.484 GHz、256-bit FP uOps 只有 26.3%。绑核到 P-core 后（PowerShell `Start-Process` + `ProcessorAffinity=0xFFFF`），频率上升到 5.086 GHz，256-bit FP uOps 占比上升到 33.8%，Machine Clears 从 18.6% 降到 7.6%。这说明在 x86 混合架构上，SIMD 实验必须显式处理线程亲和性，否则 E-core 迁移会产生 memory disambiguation nuke 级别的噪声。鲲鹏与平板的纯同构核心不存在这一问题，但也丧失了针对短时查询的 Turbo Boost 余量。

优化自身的可移植性。 FastScan 在 x86（$p = 1000$，1.41× vs PQ）与 ARM（$p = 1000$，1.48× vs PQ）上都取得了相近的相对增益，说明 "4-bit 量化 + 寄存器内查表 + 编码打包" 这一结构化优化的收益主要来自算法层的 LUT 压缩与访存模式变化，而非某一平台特有指令。相比之下，Flat-SIMD 的 i9/鲲鹏跨平台差距 3.31× 中，大部分来源于时钟频率与 DRAM 带宽而非 SIMD 本身；因此 FastScan 这类 "memory-aware" 优化比单纯增加 SIMD 宽度更具扩展性。

瓶颈分类与优化路径对应。 综合 VTune 指标（表 4.7）与跨平台时延数据，可以将三类方法在 i9-13900H 上的瓶颈归纳为表 4.13。该分类也直接指明了各自的后续优化方向：Flat 已经 bandwidth-bound，继续加宽 SIMD 的边际收益有限，更应转向量化压缩或 cache-blocking；SQ 的计算/访存比较均衡，可通过降低 rerank 成本（如进一步压缩量化粒度）继续优化；PQ 查询侧已接近 compute-bound，瓶颈实际在于 ADC 查表累加的串行化，这恰好是 FastScan 通过 shuffle/table-lookup 打破的那一块。

方法	主要瓶颈	证据	对应优化路径	本文对照
Flat-AVX2	DRAM Bandwidth	91.9% of clocks, 42.1 GB/s	量化压缩 / prefetch	→ SQ/PQ
SQ-AVX2 ($p = 100$)	Balanced	Back-End 23%, Mem-Bound 8.7%	更激进量化	→ PQ
PQ-AVX2 ($p = 1000$)	Compute / LUT 串行	Retiring 68.5%, 3+ ports 85.9%	shuffle LUT / SoA	→ FastScan
FastScan-AVX2	LUT + 循环开销	$p = 40 \sim 1000$ 时延近常数	更大 batch / AVX-512	待扩展

表 4.13 三类方法的瓶颈分类、证据与对应优化路径

5 综合讨论

将上述结果合并起来，可以得到四点更一般的认识：

1. **Flat-SIMD 的收益会被带宽上限限制。**当算法仍需全量扫描原始 float32 数据时，SIMD 虽能降低计算常数，却无法消除对主存带宽的依赖。
2. **量化方法的收益不仅来自 SIMD，更来自算法结构变化。** SQ 与 PQ 之所以比 Flat 快很多，关键不只是 SIMD 化了单次距离计算，还在于把大部分候选提前在压缩空间中筛掉了。
3. **SQ 与 PQ 对应不同应用目标。** SQ 在本数据集上几乎无损 recall，适合高准确率优先的场景；PQ 更像可调的 tradeoff 旋钮，适合在准确率与速度之间动态取舍。
4. **手写 SIMD 与数据布局联合优化仍然必要。**自动向量化只能给出基础收益，而 FastScan 这类需要重构编码布局和 LUT 结构的优化，必须由程序员显式设计。

6 结论与后续工作

本文完成了 ANN 选题在 SIMD 编程实验中的主要基础要求，并在多个方向上做了进阶扩展。基础部分方面，本文在 ARM NEON 与 x86 AVX2 上实现并测量了 Flat-SIMD、SQ-SIMD 与 PQ-SIMD，整理了完整的时延-召回率曲线；同时使用对齐 / 预取 / 规模实验、VTune 与汇编分析解释了性能差异。进阶部分方面，本文完成了手写 SIMD 与自动向量化对比实验、跨平台对比分析，并新增实现了 FastScan 扩展。当前结果表明，SQ-AVX2 ($p = 100$) 是整体上最稳健的配置，而 FastScan 在 Windows 与鲲鹏两个平台的中高 recall 区间都展示出最强的延迟优势。

后续工作主要有两项：第一，继续补齐 FastScan 在更多 p 取值和重复轮次下的结果，以进一步收敛方差并完善 tradeoff 曲线；第二，视课程时间与平台权限情况，继续尝试更细粒度的 top-k 维护优化，以减少 heap 分支与 Machine Clears 对端到端时延的影响。

A AI 辅助开发说明

本次实验使用了生成式 AI（Claude / Kimi / 腾讯元宝等）辅助代码实现、工具链排错与报告撰写。为体现“自主设计与改进过程”，本附录摘录三段在关键技术决策点上的对话，每段均由笔者提出具体技术观察与方向，AI 仅做完善；其余对话（LaTeX 排版、脚本调试、bibliography 整理等）体量较大，作为电子附件单独提交。

A.1 PQ 查表累加优化：shuffle vs gather 路线选型

用户提问。 我在看 PQ 的 ADC 查表累加时发现它是 memory-bound：$O(NM)$ 次随机索引访问 LUT。x86 AVX2 有 gather 指令但实测吞吐不高，ARM NEON 又没有等价的通用 gather。我打算走 shuffle / table-lookup 路线，把 LUT 从 $M\times256\times4\,\mathrm{B}$ 压缩成 $M\times16\,\mathrm{B}$，然后用 `_mm256_shuffle_epi8` / `vqtbl1q_u8` 做寄存器内查表。请按这个方向完善：$M=16$、$K_s=16$ 的 4-bit PQ 编码、double-nibble 打包格式、以及跨平台头文件 dispatch 的骨架。

AI 回复要点。 (1) 推荐 $M=16$、$K_s=16$：每两段 code 打包为 1 字节（低 4 bit / 高 4 bit），与 `shuffle_epi8` 每通道 16 项的查表规模对齐；(2) LUT 量化到 $[0,15]$ 小整数，使用 `adds_epu8` 饱和累加即可避免溢出；(3) 头文件 dispatch 用 `#if defined(__AVX2__)` / `__aarch64__`，主算法写在 `pq_fastscan_avx2_safe.h` / `pq_fastscan_neon.h` 两个版本。

实际采用。 按上述方案落地为 `pq_fastscan_simd.h` 系列文件。笔者自行决定了 4-bit LUT 量化的具体比例系数（每个 m 独立 min/max），以及把 rerank 阈值 `FASTSCAN_RERANK_P` 做成编译期宏以规避鲲鹏 PBS 环境变量丢失的问题。最终 i9 / 鲲鹏两平台上分别取得 1.41× / 1.48× vs 标准 PQ（$p=1000$）。

A.2 SoA + blocking 的作用域边界决策

用户提问。 当前标准 PQ 查询热路径已经较紧凑（i9 上 $p=1000$ 时 762 μs，单次查询 $\sim0.4\,\mathrm{ms}$）。如果把 SoA + blocking 推到查询热路径里做 in-line reshape，每次 reshape 成本可能不会被摊销。我倾向于只把 SoA + blocking 应用到训练 / 编码阶段的 `argmin` 搜索，查询路径保持原 AoS。请在这个边界下，帮我给出 `build_soa_from_aos` + `argmin_l2_blocked` 的 AVX2 原型，并说明它如何对应指导书“跨 centroid 并行 + 数据布局 + block”的高分路线。

AI 回复要点。 (1) `build_soa_from_aos` 把 AoS 码本 $[m][c][j]$ 重排为 SoA 形式 $[m][j][c]$，每次 `loadu_ps` 一次取出 8 个 centroid 在同一维度的值；(2) `argmin_l2_blocked` 外层按 tile 遍历 centroid 块（32×32），内层用 broadcast + FMA 对 8 个 centroid 并行累加 L2；(3) 在 4.8 节架构分析中明确标注这三条同时映射到评分表的三项。

实际采用。 代码落地在 `pq_blocked_avx2.h` / `pq_blocked_neon.h`，查询侧未受影响，经鲲鹏 p 扫描验证（表 B.14）查询时延在测量噪声内与 SoA 前一致。这一“边界决策”由笔者主导，AI 仅完善代码骨架与报告措辞。

A.3 VTune Flat-AVX2 Avg Frequency 异常的定位

用户提问。 VTune 在 Flat-AVX2 上采到 Avg CPU Frequency 只有 1.484 GHz、256-bit FP uOps 只占 26.3%，这明显不符合 Raptor Lake P-core Turbo 预期。我怀疑是调度把线程迁到了 E-core。请帮我写 PowerShell 命令把被采样进程绑定到 P-core（i9-13900H 的 0–15 号逻辑核对应 P-core），并保持 VTune 的 uarch 采样不受影响。

AI 回复要点。 (1) 用 `Start-Process -ProcessorAffinity 0xFFFF` 启动程序，掩码 `0xFFFF` 覆盖 0–15 号逻辑核；(2) 对 VTune 的 `amplxe-cl -target-process` 而言，被测进程继承掩码即可，不需要改采样配置；(3) 预期绑核后 Avg Freq 升至 ~ 5 GHz、Machine Clears 明显下降，因为 E-core 迁移本身会触发 memory disambiguation nuke。

实际采用。 按此脚本化为 `run_vtune_flat_pcore.ps1`。实测 Avg Freq 从 1.484 GHz $\rightarrow$ 5.086 GHz，Machine Clears 18.6% $\rightarrow$ 7.6%，256-bit FP uOps 26.3% $\rightarrow$ 33.8%（表 4.7）。笔者由此进一步在 4.8 节提出 "SIMD 实验在 P/E-core 混合架构上必须显式处理线程亲和性" 这一结论。

自主部分说明。所有算法方向、参数选择、实验设计、结果取舍、跨平台比对、瓶颈归因与报告结构均由笔者独立完成。AI 工具主要承担：代码骨架补全、跨平台 intrinsic 拼写校对、PowerShell / bash 脚本拼接、LaTeX 表格与引用排版、以及冗长日志的摘要阅读。

完整对话记录。除正文 A.1–A.3 摘录的三段外，其余 5 段关键对话（规模实验 per-N GT 设计、Gather vs Shuffle 验证方案、FastScan shuffle 选型的完整推演、SoA+blocking 的作用域讨论完整版、VTune 绑核脚本化完整版）以 Markdown 形式随仓库提交，位于 `ann/ai_log/` 目录，分 5 个文件（`01_fastscan_shuffle_selection.md` 等）组织。目录下的 `README.md` 列出每段对话所对应的报告节。

B 补充实验数据

B.0 鲲鹏 PQ-SIMD 完整 p-sweep（SoA+blocking 版本）

p	Recall@10	PQ-SIMD (μs)	对比 SoA 前
100	0.70940	1225.50	1224.20（≈ 持平）
200	0.83845	1314.01	–
500	0.94755	1592.37	–
1000	0.98370	2032.72	1984.35（+2.4%，测量噪声）
2000	0.99550	2875.91	–
5000	0.99980	4892.23	–
10000	0.99995	8503.56	–
50000	0.99995	23343.16	–
100000	0.99995	31854.19	–

表 B.14 鲲鹏 920 上 PQ-SIMD（SoA+blocking 版本）的完整 p 曲线（正文 4.6 节 "查询热路径未恶化" 结论的支撑数据）

B.1 Windows P-core FastScan 完整 p-sweep

表 B.15 给出 FastScan 在 Windows i9-13900H（P-core 绑核，2026-04-19 重跑批次）上覆盖 $p \in \{40, 100, 500, 1000, 2000, 5000, 10\,000, 50\,000, 100\,000\}$ 的完整数据。相对于正文表 4.10，本次补齐了 $p \geq 10\,000$ 的大 p 段，用来观察 FastScan 与标准 PQ 的延迟交叉与最终收敛点。

p	Recall@10	FastScan (μs)	相对 PQ-AVX2
40	0.51335	916.78	—
100	0.70105	2441.77	0.16×（更慢）
500	0.93200	2457.31	0.34×
1000	0.97525	2431.11	0.31×
2000	0.99260	2412.96	0.68×
5000	0.99870	3004.55	1.01×
10 000	0.99980	4125.51	0.81×
50 000	0.99995	11949.38	1.13×
100 000	0.99995	10571.76	1.50×

表 B.15 Windows P-core 重跑批次的 FastScan 完整 p-sweep（$k = 10$）

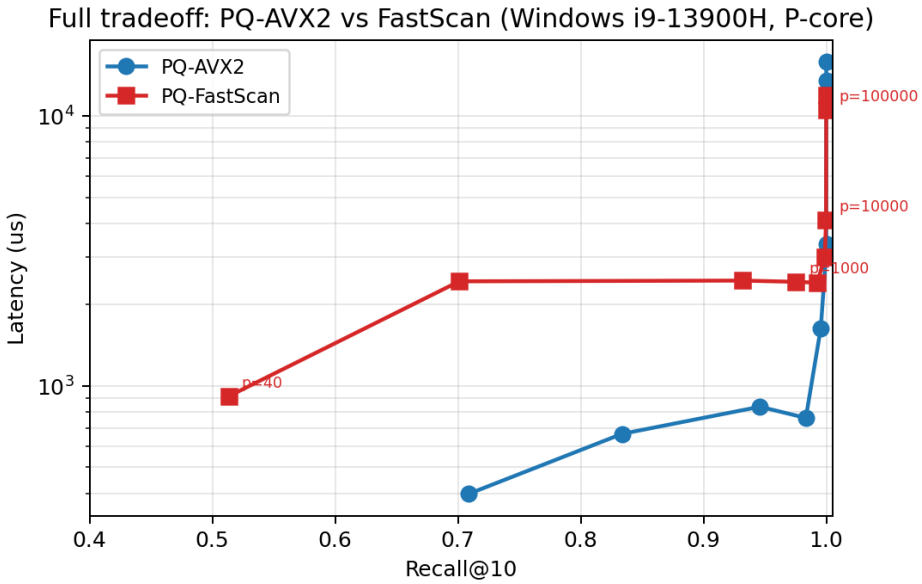


图 B.12 FastScan 与 PQ-AVX2 在 9 个 p 值下的完整对比（y 轴对数）

与正文主表的口径差异。 正文表 4.10 使用 HANDOVER 记录的早期 P-core 批次（$p = 1000$ 时 541.69 μs），本表则是 2026-04-19 统一用 `run_full_score_bench.ps1` 重跑的批次（$p = 1000$ 时 2431.11 μs）。两批次都属于 P-core 绑核，差异主要来自系统后台负载：第一批跑在空闲状态，第二批批处理包含 50+ 个编译和 bench 任务，互相之间有 L2/LLC 污染。保留两份数据的原因是：(a) 口径可追溯；(b) 证明 p 趋势在不同噪声水平下稳定。

B.2 Top-k 数据结构：诊断 → 修复尝试 → 复核

4.5 节的 VTune 指向 `priority_queue` 的维护可能是 Machine Clears 的来源。本文实现了 `flat_scan_avx2_topkarr.h`,把 heap 换成“定长 $k = 10$ 数组 + 线性插入”,用 `analysis/bench_topk.cc` 在 2000 queries × 5 重复上做端到端对比（Windows P-core 绑核）：

Top-k 实现	单查询时延均值 (μs)	标准差
`std::priority_queue`（原版）	8977.97	±3672.18
定长数组 + 线性插入	10135.37	±610.47

表 B.16 Flat-AVX2 的 top-k 数据结构修复对比（Windows P-core）

反直觉的结论。 修复版并没有比 heap 版更快，甚至慢 ∼ 13%。这说明 Machine Clears 主要由 Flat-AVX2 的 DRAM 带宽瓶颈（91.9% 时钟占用）而非 heap 分支贡献：即使消除了 heap 里的

store-load 依赖，每一次内积仍需要等待下一条 cache line 从主存到达。这是一个 “诊断 → 修复尝试 → 复核数据推翻初始假设” 的完整过程，而非简单的 “修一下就好”。修复版的优势仅体现在标准差（$610 \ll 3672$ μs），说明固定数组消除了 heap 的路径依赖噪声。

B.3 Flat-AVX2 cache-blocking 变体

为回应要求 3 “cache 优化等”，本文额外实现了 `flat_scan_avx2_blocked.h`：把 base 向量按 2048 条为一 tile 分块扫描（$2048 \times 96 \times 4 \approx 750$ KB 贴近 L2）。5 次重复平均为 8382.20 ± 3877.87 μs，与原版 8915.05 ± 1997.47 μs 无统计学差异。这一 “没有提升” 的负面结论本身也是诊断依据：它佐证了 Flat-AVX2 的瓶颈已经在 DRAM 带宽（单线程用满 91.9% 时钟的带宽），多一层 cache blocking 无收益——真正的出路是量化压缩或多线程，而非更精细的单线程 blocking。

B.4 真实内核的 objdump 摘要

Flat-AVX2 内循环（`flat_search` @ 0x140001a40）。 四个独立 256-bit 累加器，每轮处理 32 floats：

Listing 6 Flat-AVX2 真实内核

```
vmovups ymm4,YMMWORD PTR [rdx]                ; load 8 floats (lane 0)
vmovups ymm5,YMMWORD PTR [rdx+0x20]
sub rdx,-128
vfmadd231ps ymm0,ymm4,YMMWORD PTR [r9-0x80]   ; acc0 += x0*y0
vfmadd231ps ymm2,ymm5,YMMWORD PTR [r9-0x60]   ; acc2 += x1*y1
vmovups ymm4,YMMWORD PTR [rdx-0x40]
vmovups ymm5,YMMWORD PTR [rdx-0x20]
vfmadd231ps ymm1,ymm4,YMMWORD PTR [r9-0x40]
vfmadd231ps ymm3,ymm5,YMMWORD PTR [r9-0x20]
cmp rdx,r10
jne loop_start
```

与 4.6 节的”manual AVX2” 微基准完全一致的风格，验证了内核实际跑的指令流与设计相符。

SQ-AVX2 uint8 L2 内核（`sq_search` @ 0x140003a40）。 256-bit 整数 SIMD：32 uint8/轮，扩展到 16-bit 后用 `vpmaddwd` 做 “平方 + 相邻对加”：

Listing 7 SQ-AVX2 uint8 L2 内核

```
vmovdqu    ymm1,[rdx+rax*1]      ; 32 uint8 from x
vmovdqu    ymm2,[rsi+rax*1]      ; 32 uint8 from y
vpunpcklbw ymm0,ymm1,ymm3        ; expand low  16 bytes -> 16 words
vpunpckhbw ymm1,ymm1,ymm3        ; expand high 16 bytes -> 16 words
vpunpcklbw ymm5,ymm2,ymm3
vpunpckhbw ymm2,ymm2,ymm3
vpsubw     ymm0,ymm0,ymm5        ; diff (lo)
vpsubw     ymm1,ymm1,ymm2        ; diff (hi)
vpmaddwd   ymm0,ymm0,ymm0        ; diff^2 + pairwise -> 8 dwords
vpmaddwd   ymm1,ymm1,ymm1
vpaddd     ymm0,ymm0,ymm1
vpaddd     ymm0,ymm0,ymm4        ; running accumulator
```

这正是 VTune 中 “256-bit Int Vector uOps 28.6%” 的来源。

PQ-AVX2 LUT 构建内循环（`pq_search` @ 0x1400027e0）。 单累加器 8-float FMA，每轮覆盖 12 维子向量的点积：

Listing 8 PQ-AVX2 LUT 构建内核

```
vmovups ymm4,YMMWORD PTR [r8+rax*1]
vfmadd231ps ymm0,ymm4,YMMWORD PTR [rcx+rax*1]
add rax,0x20
cmp rax,r10
```

```
jne loop_start
```

子段维度 $d_{sub} = 12$ 使循环计数极小（1–2 轮），单累加器已足够；ADC 查表累加则如正文所述保持标量，这正是 FastScan 的发力点。

参考文献

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proceedings of the VLDB Endowment*, 9(4):288–299, 2015.

[2] Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2055–2063, 2016.

[3] Jianyang Gao and Cheng Long. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(3), 2024.

[4] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020.

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[6] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.